

Concepte Cheie pentru Examen 2

exec, dup/dup2, Pipe-uri si I/O

Sisteme de Operare - Tompea Maria-Daria

Cuprins

1	Ce este exec()?	2
1.1	De ce avem nevoie de exec()?	2
1.2	Regulile de Aur ale lui exec()	2
1.3	Exemplu de baza: exec() care reuseste	2
1.4	Exemplu: exec() care esueaza	3
1.5	Pattern-ul Clasic: fork() + exec()	3
1.6	Familia exec() – Cele 6 Variante	4
2	Imaginile in Oglinda: dup() si dup2()	6
2.1	Ce este un File Descriptor?	6
2.2	dup(oldfd) – Dubleaza fara sa alegi numarul	6
2.3	dup2(oldfd, newfd) – Dubleaza SI alegi numarul	7
2.4	dup2() + exec() = Redirectarea I/O a unui Program Extern	7
3	Pipe-uri Manuale cu pipe(), fork(), dup2(), exec()	8
4	Functiile I/O: open, fopen, popen	9
4.1	open() – Nivel jos (System Call)	9
4.2	fopen() – Nivel inalt (Biblioteca C)	10
4.3	popen() – Deschide un Sub-Proces ca un Fisier	11
5	Comparatia Master: Toate Functiile	12
5.1	Functiile I/O (Deschizatori si Inchizatori)	12
5.2	Mecanismele de Baza: Procese si Redirectare	12
5.3	Cum alegi ce sa folosesti – Ghid Rapid	13
6	Rezumat Final pentru Examen	13

1 Ce este exec()?

Definitie: exec()

`exec()` nu creeaza un proces nou. In schimb, **inlocuieste complet procesul curent** cu un program nou. PID-ul ramane acelasi, dar tot codul, memoria si stack-ul sunt sterse si inlocuite cu noul program.

1.1 De ce avem nevoie de exec()?

`fork()` creeaza o copie a procesului tau. Dar adesea nu vrei sa rulezi o copie a ta – vrei sa rulezi un program *complet diferit*, de exemplu `ls`, `grep`, sau `gcc`. Aici intervine `exec()`.

Fluxul clasic este:

1. `fork()` – creezi un copil (o copie a ta)
2. In copil: `exec()` – copilul isi “schimba creierul” cu un alt program
3. In parinte: `wait()` – parintele asteapta copilul sa termine

Analogie din viata reala

Imagineaza-ti un runner pe o pista. Cand se apeleaza `exec()`, runner-ul:

- **Pastreaza:** acelasi corp (PID), fisierele deschise, variabilele de mediu
- **Pierde instant:** tot creierul (codul), toata memoria, toate variabilele
- **Capata:** creierul, codul si memoria unui cu totul alt program

Continua sa alerge din acelasi punct, dar acum este o persoana complet diferita.

1.2 Regulile de Aur ale lui exec()

De retinut

1. **Nu returneaza niciodata daca reuseste.** Daca `exec()` functioneaza, tot codul tau de dupa apel este sters din memorie. Liniile de dupa `exec()` pur si simplu nu vor mai exista.
2. **Returneaza doar daca esueaza.** Daca returneaza `-1` (de exemplu, programul nu a fost gasit), programul tau original continua sa ruleze pentru a putea trata eroarea.

1.3 Exemplu de baza: exec() care reuseste

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("Sunt procesul original. PID: %d\n", getpid());
6     printf("Acum apelez exec()... pe mine ma puteti uita!\n");
7
8     // Inlocuim COMPLET acest proces cu programul "ls -l"
9     execl("/bin/ls", "ls", "-l", NULL);
```

```

10
11 //
12
13 // ATENTIE: aceasta linie NU se va executa niciodata daca exec()
14 // reuseste!
15 // Daca o vedem in output, inseamna ca exec() a ESUAT.
16 //
17
18 perror("exec() a esuat!");
19 return 1;
20 }

```

Listing 1: exec() inlocuieste procesul cu /bin/ls

Output asteptat: In loc de orice mesaj de la programul tau, vei vedea pur si simplu listarea directorului curent (rezultatul lui `ls -l`), din moment ce procesul a fost inlocuit complet.

1.4 Exemplu: exec() care esueaza

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("Incerc sa rulez un program care nu exista...\n");
6
7     execl("/bin/programcare_nu_exista", "programcare_nu_exista", NULL
8 );
9
10    // Aceasta linie VA rula, pentru ca exec() a esuat
11    perror("Eroare exec"); // Va afisa: "Eroare exec: No such file or
12    // directory"
13    printf("exec() a esuat, sunt inca in viata!\n");
14
15    return 1;
16 }

```

Listing 2: Ce se intampla cand exec() nu gaseste programul

1.5 Pattern-ul Clasic: fork() + exec()

Acesta este **cel mai important pattern** din tot capitolul. Asa lanseaza un shell un program nou:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t pid = fork(); // Pasul 1: clonam procesul
8
9     if (pid < 0) {
10         perror("fork esuat");

```

```
11     exit(1);
12 }
13
14 if (pid == 0) {
15     //          COPIL
16
17     // Pasul 2: copilul isi inlocuieste creierul cu "ls -la"
18     printf("[COPIL] PID: %d, ma transform in ls...\n", getpid());
19     execl("/bin/ls", "ls", "-la", NULL);
20     // Daca ajungem aici, exec() a esuat
21     perror("[COPIL] exec() a esuat");
22     exit(1); // Important: copilul trebuie sa iasa daca exec
23     esueaza!
24 } else {
25     //          PARINTE
26
27     // Pasul 3: parintele asteapta copilul sa termine
28     printf("[PARINTE] PID: %d, am creat copilul %d\n", getpid(),
29     pid);
30     int status;
31     wait(&status);
32     printf("[PARINTE] Copilul a terminat.\n");
33 }
34
35 return 0;
36 }
```

Listing 3: Pattern-ul clasic fork + exec + wait

Atentie!

Gresala clasica: Daca `exec()` esueaza in copil si nu apelezi `exit()`, copilul va continua sa execute codul parintelui! Ai putea ajunge cu doua procese care fac acelasi lucru – un bug greu de depanat.

1.6 Familia `exec()` – Cele 6 Variante

Nu intra in panica la numele lor. Literele iti spun **exact** cum se transmit argumentele:

Funcție	Ce înseamnă literalmente	Cum se apelează
<code>execl</code>	list	Argumente ca lista, separate prin virgula, terminate cu NULL
<code>execv</code>	vector	Argumente ca array de string-uri
<code>execlp</code>	list + path	Ca <code>execl</code> , dar caută automat în PATH
<code>execvp</code>	vector + path	Ca <code>execv</code> , dar caută automat în PATH
<code>execle</code>	list + env	Ca <code>execl</code> , dar poți specifica variabile de mediu custom
<code>execve</code>	vector + env	Ca <code>execv</code> , dar poți specifica variabile de mediu custom

```

1 #include <unistd.h>
2
3 //      execl: cale absoluta + lista de argumente
4
5 execl("/bin/ls", "ls", "-l", "-a", NULL);
6 //      ^ cale      ^ argv[0]  ^ arg ^ OBLIGATORIU NULL la final!
7
8 //      execv: cale absoluta + array
9
10 char *args[] = {"ls", "-l", "-a", NULL}; // array-ul TREBUIE terminat
11 //      cu NULL
12 execv("/bin/ls", args);
13
14 //      execlp: fara cale absoluta -- caută singur în PATH
15
16 execlp("ls", "ls", "-l", NULL);
17 //      Nu trebuie să știi că ls se află în /bin/ls!
18
19 //      execvp: array + caută în PATH
20
21 char *args2[] = {"ls", "-l", NULL};
22 execvp("ls", args2);

```

Listing 4: Exemple pentru variantele `exec()`**De reținut**

Sfat practic: La examen, `execlp` și `execvp` sunt cele mai comune, deoarece nu trebuie să cunoști calea absolută a programului. Dacă știi să folosești `execl` și `execv`, le știi pe toate.

2 Imaginile in Oglinda: dup() si dup2()

Definitie: dup() si dup2()

dup() si dup2() **dubleaza un file descriptor**. Dupa apel, doua numere diferite de file descriptor vor pointa la exact acelasi fisier, pipe, sau device.

2.1 Ce este un File Descriptor?

Un file descriptor (FD) este pur si simplu un **numar intreg** pe care kernelul il foloseste ca un “bilet de intrare” catre un fisier sau resursa. Fiecare proces primeste automat 3 la start:

Numar FD	Nume	Ce este
0	STDIN	Intrarea standard (tastatura)
1	STDOUT	Iesirea standard (terminalul/ecranul)
2	STDERR	Iesirea de erori (tot terminalul, dar separat)

Analogie din viata reala

Gandeste-te la file descriptorii ca la **numerele de la vestiare**. FD 1 (stdout) este cuierul numarul 1. In mod normal, pe cuierul 1 atarni “terminalul” (ecranul). Dupa dup2(fisier_fd, 1), pe cuierul 1 atarni “fisierul pe disc”. Orice program care scrie la cuierul 1 va scrie acum in fisier, fara sa stie!

2.2 dup(oldfd) – Dubleaza fara sa alegi numarul

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <fcntl.h>
4
5 int main() {
6     // Deschidem un fisier, obtinem FD-ul lui (sa zicem 3)
7     int fd = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
8     if (fd < 0) { perror("open"); return 1; }
9
10    printf("FD-ul fisierului: %d\n", fd); // Probabil 3
11
12    // dup() ia primul FD disponibil (probabil 4) si il face copie a
    lui fd
13    int fd_copie = dup(fd);
14    printf("FD-ul copiat: %d\n", fd_copie); // Probabil 4
15
16    // Acum FD 3 si FD 4 pointeaza la ACELASI fisier!
17    write(fd, "Scriere prin FD 3\n", 18);
18    write(fd_copie, "Scriere prin FD 4\n", 18);
19
20    // Fisierul va contine AMBELE linii!
21    close(fd);
22    close(fd_copie);
23    return 0;
24 }
```

Listing 5: dup() creeaza o copie la primul FD disponibil

2.3 dup2(oldfd, newfd) – Dubleaza SI alegi numarul

dup2 este **versiunea puternica**. Tu specifici exact ce numar vrei. Daca newfd era deja deschis, dup2 il inchide automat mai intai.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <fcntl.h>
4
5 int main() {
6     // Deschidem un fisier pentru scriere
7     int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
8     if (fd < 0) { perror("open"); return 1; }
9
10    printf("Acest mesaj apare PE ECRAN (stdout inca = terminal)\n");
11
12    // dup2(fd, 1) face ca FD 1 (stdout) sa pointeze la fisierul
    nostru.
13    // FD 1 era inainte terminalul -- acum este fisierul!
14    dup2(fd, 1); // stdout-ul este REDIRECTAT la output.txt
15
16    // Inchidem FD-ul original -- nu mai avem nevoie de el,
17    // avem deja FD 1 care pointeaza la fisier
18    close(fd);
19
20    // printf scrie la stdout (FD 1) -- care acum este fisierul!
21    printf("Acest mesaj ajunge in OUTPUT.TXT, nu pe ecran!\n");
22
23    return 0;
24 }
```

Listing 6: dup2() redirecteaza stdout catre un fisier

2.4 dup2() + exec() = Redirectarea I/O a unui Program Extern

Acesta este **cel mai important use-case** al lui dup2: redirectezi I/O-ul unui copil *inainte* ca el sa se transforme cu exec().

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/wait.h>
6
7 int main() {
8     // Pasul 1: Deschidem fisierul unde vrem sa mearga outputul
9     int fd = open("ls_output.txt", O_WRONLY | O_CREAT | O_TRUNC,
10    0644);
11    if (fd < 0) { perror("open"); exit(1); }
12
13    pid_t pid = fork(); // Pasul 2: Clonam procesul
14
15    if (pid == 0) {
```

```

15      //          COPIL

16      // Pasul 3: Redirectam stdout-ul COPILULUI catre fisier
17      dup2(fd, 1); // FD 1 (stdout) acum pointeaza la ls_output.txt
18      close(fd);   // Inchidem FD-ul original, nu mai avem nevoie

19      // Pasul 4: Transformam copilul in "ls -la"
20      // ls va scrie la stdout (FD 1)... care acum este fisierul!
21      execl("/bin/ls", "ls", "-la", NULL);
22      perror("exec esuat");
23      exit(1);
24
25
26  } else {
27      //          PARINTE

28      close(fd); // Parintele inchide si el FD-ul fisierului
29      wait(NULL);
30      printf("Gata! Outputul lui ls a mers in ls_output.txt\n");
31  }
32
33  return 0;
34  }

```

Listing 7: Redirectam stdout-ul lui ls intr-un fisier

Intrebare tip Examen

Intrebare clasica: “De ce inchidem fd in parinte dupa fork()?”

Raspuns: Dupa fork(), atat parintele cat si copilul au o copie a lui fd. Daca parintele nu il inchide, fisierul ramane deschis in parinte chiar si dupa ce copilul termina. Trebuie ca **toti** deschizatorii unui FD sa il inchida pentru ca fisierul sa fie cu adevarat inchis.

3 Pipe-uri Manuale cu pipe(), fork(), dup2(), exec()

Acum combinam tot ce stim. Asa functioneaza comanda `ls -la | grep .c` in shell:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7      int pipefd[2]; // pipefd[0] = capatul de citire, pipefd[1] =
8                      // capatul de scriere
9
10     if (pipe(pipefd) < 0) { perror("pipe"); exit(1); }
11
12     //          Primul copil: va rula "ls -la"
13
14     pid_t pid1 = fork();
15     if (pid1 == 0) {

```

```

14 // Redirectam stdout (1) catre capatul de SCRIERE al pipe-
    ului
15 dup2(pipefd[1], 1); // stdout -> pipe scriere
16 close(pipefd[0]); // copilul 1 nu citeste din pipe
17 close(pipefd[1]); // inchidem originalul (avem deja FD 1 =
    pipe)
18
19 execl("/bin/ls", "ls", "-la", NULL);
20 perror("exec ls esuat"); exit(1);
21 }
22
23 // Al doilea copil: va rula "grep .c"
24
25 pid_t pid2 = fork();
26 if (pid2 == 0) {
27     // Redirectam stdin (0) catre capatul de CITIRE al pipe-ului
28     dup2(pipefd[0], 0); // stdin -> pipe citire
29     close(pipefd[1]); // copilul 2 nu scrie in pipe
30     close(pipefd[0]); // inchidem originalul (avem deja FD 0 =
        pipe)
31
32     execl("/bin/grep", "grep", ".c", NULL);
33     perror("exec grep esuat"); exit(1);
34 }
35
36 // PARINTELE: inchide ambele capete si asteapta
37
38 // Important! Daca parintele nu inchide pipe-ul, grep va astepta
    la infinit
39 // pentru mai multa data (pipe-ul nu va fi niciodata "inchis" din
    perspectiva lui)
40 close(pipefd[0]);
41 close(pipefd[1]);
42
43 // Asteptam ambii copii
44 wait(NULL);
45 wait(NULL);
46
47 printf("Comanda pipe a terminat!\n");
48 return 0;
49 }

```

Listing 8: Implementarea manuala a lui `ls -la | grep .c`

4 Functiile I/O: open, fopen, popen

4.1 open() – Nivel jos (System Call)

```

1 #include <fcntl.h>
2 #include <unistd.h>
3

```

```

4 int main() {
5     // open() returneaza un int (file descriptor)
6     // O_RDONLY = deschide doar pentru citire
7     // O_WRONLY = deschide doar pentru scriere
8     // O_CREAT  = creeaza fisierul daca nu exista
9     // O_TRUNC  = sterge continutul existent
10    // 0644     = permisiunile (rw-r--r--)
11
12    int fd = open("fisier.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
13    if (fd < 0) { perror("open esuat"); return 1; }
14
15    // Scriem cu write() -- nu cu fprintf sau printf!
16    char *mesaj = "Salut din open()!\n";
17    write(fd, mesaj, 18);
18
19    close(fd); // Nu fclose()! Folosim close() pentru file
20    // descriptori.
21    return 0;
22 }

```

Listing 9: open() – lucreaza direct cu file descriptori

4.2 fopen() – Nivel inalt (Biblioteca C)

```

1 #include <stdio.h>
2
3 int main() {
4     // fopen() returneaza un pointer FILE*
5     // "w" = deschide pentru scriere (creeaza/trunccheaza)
6     // "r" = deschide pentru citire
7     // "a" = deschide pentru append (adauga la sfarsit)
8
9     FILE *f = fopen("fisier.txt", "w");
10    if (f == NULL) { perror("fopen esuat"); return 1; }
11
12    // Scriem cu fprintf() sau fputs() -- nu cu write()!
13    fprintf(f, "Salut din fopen()! Numarul meu favorit: %d\n", 42);
14
15    // fclose() face doua lucruri:
16    // 1. Da flush la buffer (trimite ce e in memorie pe disc)
17    // 2. Inchide file descriptorul
18    fclose(f);
19    return 0;
20 }

```

Listing 10: fopen() – lucreaza cu buffere, mai usor de folosit

De retinut

De ce exista buffer-ul la fopen/fclose?

Scriul pe disc este *lent*. Cand apelezi `fprintf(f, "x")`, biblioteca C nu scrie imediat pe disc. Acumuleaza datele intr-un buffer in memorie si le scrie toate odata cand buffer-ul e plin sau cand apelezi `fclose()/fflush()`. Asta face I/O-ul mult mai rapid.

`open() + write()` nu au acest buffer – fiecare `write()` merge direct la kernel. Mai

lent, dar mai predictibil.

4.3 popen() – Deschide un Sub-Proces ca un Fisier

`popen()` este o **scurtatura de lux**: ruleaza o comanda si iti ofera un pipe catre ea, gata de folosit, fara sa faci tu `fork()`/`exec()` manual.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     // popen() lanseaza "ls -la" ca proces copil si returneaza
6     // un FILE* conectat la stdout-ul acelui proces
7     FILE *cmd = popen("ls -la", "r");
8     if (cmd == NULL) { perror("popen esuat"); return 1; }
9
10    char linie[256];
11    printf("Outputul comenzii ls -la:\n");
12    printf("
13
14    // Citim linie cu linie din outputul comenzii
15    while (fgets(linie, sizeof(linie), cmd) != NULL) {
16        printf(" >> %s", linie);
17    }
18
19    // pclose() face doua lucruri:
20    // 1. Inchide stream-ul (ca fclose)
21    // 2. Asteapta copilul sa termine (ca wait)
22    int status = pclose(cmd);
23    printf("
24
25    \n");
26    printf("Comanda a terminat cu statusul: %d\n", status);
27
28    return 0;
29 }
```

Listing 11: `popen()` – citim outputul comenzii "ls -la"

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     // "w" = scriem catre stdin-ul comenzii (in loc sa citim outputul
6     // )
7     FILE *cmd = popen("sort", "w");
8     if (cmd == NULL) { perror("popen esuat"); return 1; }
9
10    // Trimitem linii dezordonate catre "sort"
11    fprintf(cmd, "banana\n");
12    fprintf(cmd, "mar\n");
13    fprintf(cmd, "afina\n");
14    fprintf(cmd, "zmeura\n");
15 }
```

```

15 // pclose() trimite datele, asteapta sort sa termine si afiseaza
    rezultatul
16 pclose(cmd); // sort va printa: afina, banana, mar, zmeura
17 return 0;
18 }

```

Listing 12: popen() in modul scriere – trimitem date catre o comanda

Atentie!

Limitarea lui popen(): Pipe-ul creat de popen() este **unidirectional**. Fie citesti outputul comenzii ("r"), fie ii trimiti input ("w"). Nu poti face ambele in acelasi timp cu un singur popen(). Daca ai nevoie de comunicare bidirectionala, trebuie sa faci manual: pipe() + fork() + dup2() + exec().

5 Comparatia Master: Toate Functiile

5.1 Functiile I/O (Deschizatori si Inchizatori)

Functie	Ce neaza	retur-	Nivel	Cum neaza	functio-	Cum inchizi
open()	int (FD)		Jos (Syscall)	Direct la fisier/de- vice. Fara buffer.		close()
fopen()	FILE * (Stream)		Inalt (Lib C)	Buffer in memorie, apoi pe disc.		fclose()
popen()	FILE * (Stream)		Inalt (Wra- pper)	Pipe catre un pro- ces copil ascuns.		pclose()

5.2 Mecanismele de Baza: Procese si Redirectare

Functie	Scop principal	Comportamentul Tricky la Exa- men
fork()	Cloneaza procesul in Parinte si Copil	Returneaza 0 copilului, PID-ul copilu- lui parintelui.
exec()	Suprascrie procesul curent cu un pro- gram nou	Nu returneaza niciodata daca reuseste. PID-ul ramane identic.
dup2()	Forteaza un FD sa mirreze alt FD	Esential pentru redirectare. Daca newfd e deschis, il inchide automat.
wait() / waitpid()	Parintele se pau- zeaza pana copilul termina	Uitand de el, creezi procese Zombie. WNOHANG nu blocheaza.

5.3 Cum alegi ce sa folosesti – Ghid Rapid

Ghid de decizie pentru examen

Q: Vrei **performanta/buffering** pentru citit/scrie fisiere simple?

A: $\Rightarrow$ `fopen()` / `fclose()`

Q: Vrei sa **rulezi o comanda externa rapid** si sa ii citesti outputul?

A: $\Rightarrow$ `popen()` / `pclose()`

Q: Vrei **control fin, securitate, pipe bidirectional, sau mai multi copii?**

A: $\Rightarrow$ `pipe()` + `fork()` + `dup2()` + `exec()` manual

Q: Vrei sa **redirectezi inputul/outputul** unui program extern?

A: $\Rightarrow$ `dup2(fd, 0)` pentru stdin, `dup2(fd, 1)` pentru stdout, inainte de `exec()`

6 Rezumat Final pentru Examen

Regulile de Aur – Invata-le pe de rost

1. `exec()` **nu creeaza** un proces nou – il **inlocuieste** pe cel curent.
2. `exec()` nu returneaza niciodata daca reuseste. Codul de dupa = mort.
3. **Litele** din familia `exec`: **l** = lista, **v** = vector/array, **p** = cauta in PATH.
4. `dup2(oldfd, newfd)` face ca `newfd` sa pointeze la ce pointeaza `oldfd`.
5. `dup2(pipefd[1], 1)` = redirecteaza stdout-ul la pipe (scriere).
6. `dup2(pipefd[0], 0)` = redirecteaza stdin-ul la pipe (citire).
7. Dupa `fork()`, **inchide** capetele de pipe neutilizate in **ambele** procese!
8. `popen()` = scurtatura pentru pipe simplu. `pclose()` = `fclose()` + `wait()`.
9. `fclose()` = flush buffer + inchide FD. `close()` = doar inchide FD.
10. `open()` / `close()` = nivel jos, fara buffer. `fopen()` / `fclose()` = nivel inalt, cu buffer.

Mult succes la examen!